

Coursework -Large Scale Data Engineering

CIRENYANGZHEN (2679375)

Part 1

1. Selection of AWS Services

1.1 Amazon CloudFront: To enable global access, I add CloudFront to cache front-end web pages and static resources. When a user accesses the website, the request is automatically routed to the nearest edge node. This accelerates global access and provide low-latency and highly available access to the site worldwide. Meanwhile, caching reduces the frequency of access to S3 and EC2, significantly lowering request and data transfer costs.

1.2 Application Load Balancer: Application Load Balancer is deployed in public subnets of two Availability Zones, serving as the unified entry point for "ArtAI." It receives requests from CloudFront and distributes traffic on demand to EC2 instances in the private subnets. Through cross-Availability Zone deployment and health checks, ALB can automatically switch traffic to healthy instances when an instance or Availability Zone fails, ensuring high availability. Furthermore, ALB prevents users from directly accessing EC2, improving overall security and controllability.

1.3 Amazon EC2: Amazon EC2 is used to deploy the backend applications for "ArtAI," which handles user uploads, business logic, and calls to the SageMaker Endpoint. The application servers are deployed in private subnet 1 and 2 within Availability Zone A and B to avoid direct exposure to the public network, thus improving security and availability. EC2 instances can be horizontally scaled according to load, and continuous service operation is ensured through Auto Scaling and health checks. Moreover, EC2 accesses input and output S3 Buckets through S3 Gateway Endpoint, eliminating the need for public network access, which help to reduce costs and improving data security.

1.4 Amazon SageMaker Endpoint: The Amazon SageMaker Endpoint is used to host the pre-trained image-generation model for ArtAI. It is deployed in a private subnet within the VPC, allowing access only from the internal EC2 instances. EC2 sends user-uploaded images to the endpoint over the internal network, receives the generated output, and writes the results to the S3 output bucket. Because all traffic stays inside the VPC, the system achieves higher security, lower latency, and cost-effective. The reason why I chose SageMaker Endpoint is that it provides a fully managed, real-time inference service for trained models. It supports automatic scaling, high-performance instance options, and strict network access control, making it ideal for ArtAI's secure and efficient image-generation requirements.

1.5 Amazon S3: Amazon S3 provides object storage, the input bucket stores raw images uploaded by users and the output bucket stores images generated by the AI model. Moreover, S3 can automatically replicates data across Availability Zone A and B, ensuring storage reliability. To meet

the need for automatic backup and recovery of older versions, S3 has version control enabled for both buckets. Each time a new version of a file is uploaded, the older version is automatically saved, thus eliminating the need for an additional backup system for backup and recovery.

1.6 Amazon S3 Gateway Endpoint: It provides a private connection from the VPC to S3, allowing EC2 instances to securely access S3 without internet access, which improves security and efficiency.

1.7 AWS CloudWatch, CloudWatch Alarm and Amazon SNS: AWS CloudWatch monitors key operational metrics across the entire application. The architecture is configured with CloudWatch's daily request count monitoring feature to track the number of API requests received by the ALB. When the daily request count exceeds 10,000, a corresponding CloudWatch Alarm is automatically triggered. The alert system then sends an email alert to administrators via Amazon SNS, enabling operations personnel to be promptly informed of system traffic surges. This monitoring and notification mechanism improves application maintainability and operational reliability.

In summary, ArtAI's architecture is a collaborative pipeline: user requests are accelerated by CloudFront and directed to ALB; ALB distributes traffic to a private EC2 cluster; EC2 acts as the processing hub, securely calling SageMaker for AI inference through internal network security, and exchanging data with S3, which has version control capabilities, through a dedicated channel; the entire process is monitored by CloudWatch, and alerts are sent via SNS when traffic is abnormal.

2. AWS Architecture Design for ArtAI

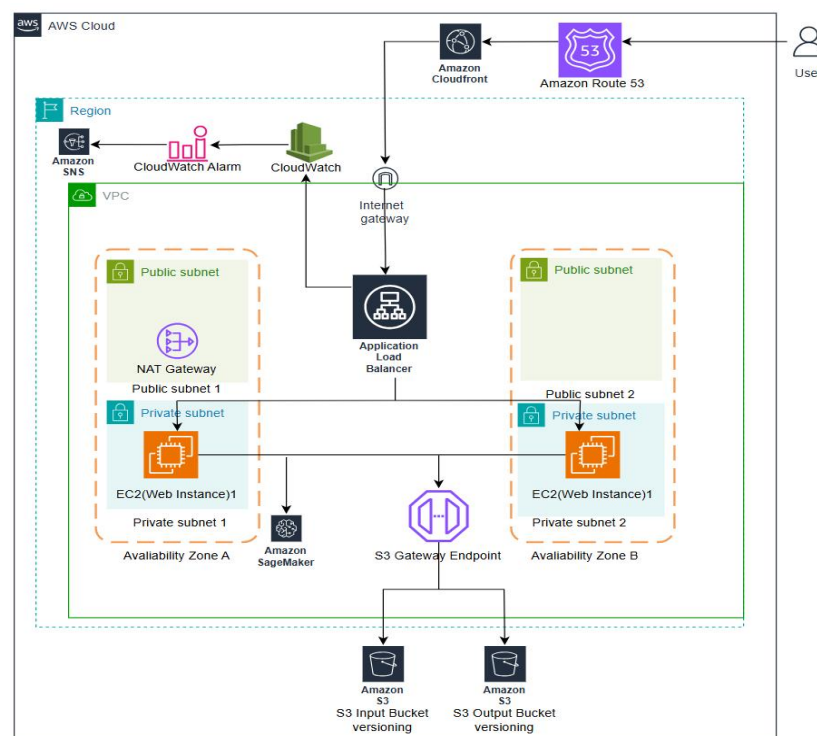


Fig.1. 1

Part 2

1. Task A

The WordFreq is a distributed text processing system built using the AWS SDK for Go. It can process uploaded text files and record the 10 most frequent words in each file. In Task A, I completed the installation and configuration of all necessary AWS resources and verified that the WordFreq application runs correctly in the AWS Academy Learner Lab environment.

1.1 Environment Setup

First, I configured and launched a development EC2 instance named “wordfreq-dev” according to the instructions in the “README.txt” file, and installed all required software and dependencies on the instance.

1.2 Installation Verification

After uploading a test file to the upload bucket, I copied it to the “ciren yangzhen-processing” bucket. The computed top 10 words were successfully written to the DynamoDB table, and a result message was published to the results SQS queue. This confirms that the workflow is functioning correctly.

1.3 WordFreq Application Workflow

The workflow of the WordFreq system is as follows:

1.3.1 File Upload: Users upload text files to the upload bucket, and the files are copied to the processing bucket. When a new file appears in the processing bucket, S3 will automatically send an event notification to the wordfreq-jobs SQS queue.

1.3.2 File Processing: the WordFreq running on the EC2 instance continuously polls the job queue. When a task arrives, the instance process reads the S3 object location from the message, then downloads the text file from the processing bucket, calculates the word frequencies, and finally selects the 10 most frequent words.

1.3.3 Storing Results: After processing, the application writes the results to the wordfreq DynamoDB table and sends a message to the wordfreq-results queue.

1.3.4 Retrieving Results: Finally, users can retrieve the processing results by querying the DynamoDB table or polling the results queue.

1.4 Architecture Diagram

The specific architecture of wordfreq is shown in Fig.2 below.

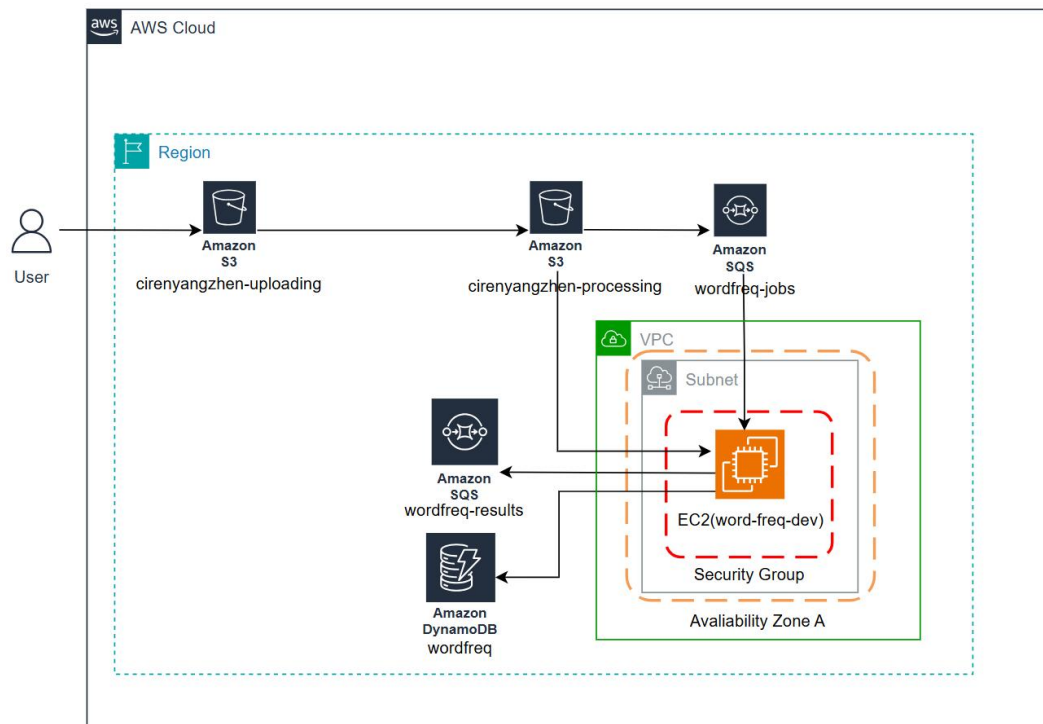


Fig.2. 1

2. Task B

The current WordFreq application architecture is based on a single EC2 instance. With the artificially induced delay, each file takes 10–20 seconds to process, so the single instance can only handle files sequentially. Its processing capacity is limited and cannot cope with sudden large file uploads. Additionally, if this instance fails, the entire service will be interrupted. Therefore, to efficiently handle the dynamically changing workload, according to the task requirements, I designed and implemented Auto-scaling in Task B.

2.1 Scaling Metric Selection and Justification

Since WordFreq is a queue-driven application, traditional metrics like CPU utilization are not suitable as auto-scaling triggers. After comparative analysis, the conclusions are as follows:

2.1.1 CPU Utilization: Although CloudWatch natively supports this metric, the application spends most of the time waiting for tasks, resulting in low CPU usage. Therefore, it cannot accurately reflect system load and is unsuitable as a scaling basis.

2.1.2 SQS Queue Message Count: This metric directly reflects the number of pending tasks in the system and can accurately represent the current workload. Although it requires custom monitoring configuration, it is the most appropriate metric.

2.1.3 Age of Oldest Message in Queue: This metric can reflect task waiting time but may be affected by a single long-running task, so it is not suitable as the primary metric.

Based on the above analysis, the ApproximateNumberOfMessagesVisible metric of the SQS queue is chosen as the metric for auto-scaling. This metric provides a direct and reliable reflection of pending tasks.

2.2 Specific Policy Configuration

Based on the above design principles, the scaling policies are configured as follows:

2.2.1 Scale-out Policy: To cope with task accumulation, a scale-out policy is designed. This threshold provides a buffer for burst tasks while avoiding false triggers due to short-term peaks. The specific action is to add 1 instance and set a 120-second cooldown to meet the minimum interval requirement and allow new instance startup time.

2.2.2 Scale-in Policy: To optimize resource usage, a scale-in policy is designed. When the queue message count remains ≤ 2 for 5 minutes, scaling in is triggered. The lower threshold ensures scaling only occurs when the queue is nearly empty, and the 5-minute duration prevents premature termination. The specific action is to terminate the most recently launched instance, with a 180-second cooldown to avoid oscillation caused by borderline load fluctuations.

2.2.3 Capacity Boundaries: To ensure continuous service availability, I set the Minimum capacity as 1 instance, hile considering the Learner Lab limit, the Maximum capacity is set to 5 instances. The desired capacity was initially set to 1, dynamically adjusted by Auto Scaling according to policies.

2.3 Implementation Steps

2.3.1 Create AMI: Create an AMI image wordfreq-dev-backup from the wordfreq-dev instance.

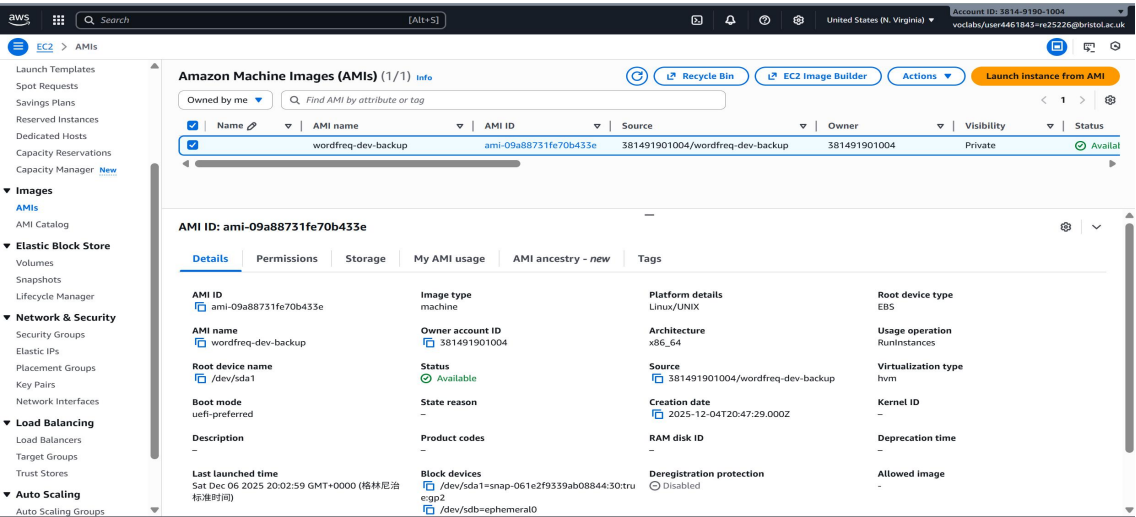


Fig.2. 2

2.3.2 Configure Launch Template: Create a launch template, selecting the previously created AMI.

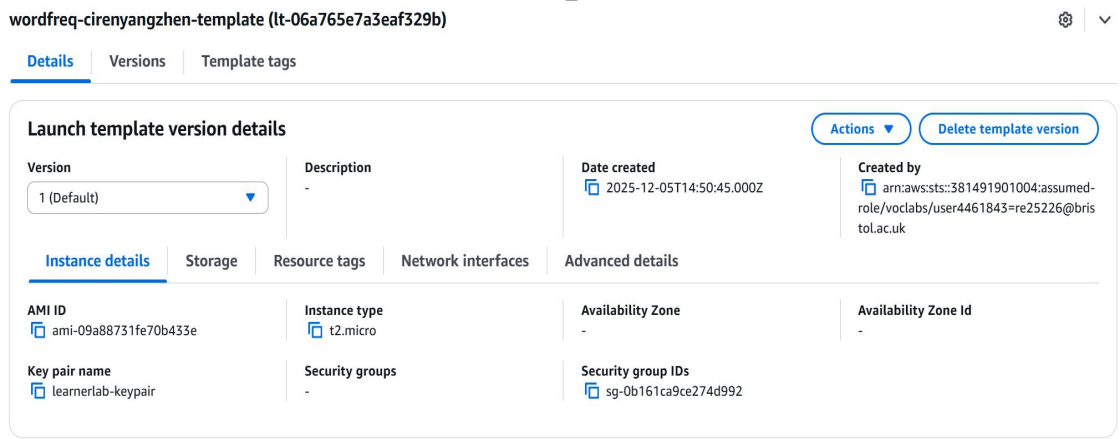


Fig.2. 3

2.3.3 Create Auto Scaling Group: Create an Auto Scaling Group cirenyangzhen-asg using the launch template. Select the default VPC and for high availability.

Auto Scaling group: cirenyangzhen-asg

cirenyangzhen-asg Capacity overview

arn:aws:autoscaling:us-east-1:381491901004:autoScalingGroup:c9456e9e-3c52-42a2-ae56-0eea2052e6b8:autoScalingGroupName/cirenyangzhen-asg

Desired capacity	Scaling limits (Min - Max)	Desired capacity type	Status
1	1 - 5	Units (number of instances)	-

Date created
Fri Dec 05 2025 15:05:04 GMT+0000 (格林尼治标准时间)

Launch template

Launch template

lt-06a765e7a3eaf329b

wordfreq-cirenyangzhen-template

Version

1

Description

-

View details in the launch template console

AMI ID

ami-09a88731fe70b433e

Security groups

-

Storage (volumes)

-

Instance type

t2.micro

Security group IDs

sg-0b161ca9ce274d992

Key pair name

learnerlab-keypair

Owner

arn:aws:sts::381491901004:assumed-role/voclabs/user4461843=re25226@bristol.ac.uk

Create time

Fri Dec 05 2025 14:50:45 GMT+0000 (格林尼治标准时间)

Request Spot Instances

No

Fig.2. 4

2.3.4 Create CloudWatch Alarms: At first, I created scale-out alarm cirenyangzhen-first with the metric specified in the Fig 2.5.

Details

Name

cirenyangzhen-first

Type

Metric alarm

Description

No description

State

OK

Threshold

ApproximateNumberOfMessagesVisible > 10 for 1 datapoints within 1 minute

Actions

Actions enabled

Last state update

2025-12-06 19:45:41 (UTC)

Namespace

AWS/SQS

Metric name

ApproximateNumberOfMessagesVisible

QueueName

wordfreq-jobs

Statistic

Sum

Period

1 minute

Datapoints to alarm

1 out of 1

Missing data treatment

Treat missing data as missing

Percentiles with low samples

evaluate

ARN

arn:aws:cloudwatch:us-east-1:381491901004:alarm:cirenyangzhen-first

View EventBridge rule

Fig.2. 5

Then I created scale-in alarm cirenyangzhen-second with the metric specified in the Fig 2.6.

Details

Name

cirenyangzhen-second

Type

Metric alarm

Description

No description

State

In alarm

Threshold

ApproximateNumberOfMessagesVisible <= 2 for 1 datapoints within 5 minutes

Actions

Actions enabled

Last state update

2025-12-06 19:49:29 (UTC)

Namespace

AWS/SQS

Metric name

ApproximateNumberOfMessagesVisible

QueueName

wordfreq-jobs

Statistic

Sum

Period

5 minutes

Datapoints to alarm

1 out of 1

Missing data treatment

Treat missing data as missing

Percentiles with low samples

evaluate

ARN

arn:aws:cloudwatch:us-east-1:381491901004:alarm:cirenyangzhen-second

View EventBridge rule

Fig.2. 6

2.3.5 Create Auto Scaling Policies: Attach dynamic scaling policies to the above alarms. Due to WordFreq’s fluctuating task volume, a step scaling policy is chosen to smoothly add or remove instances.

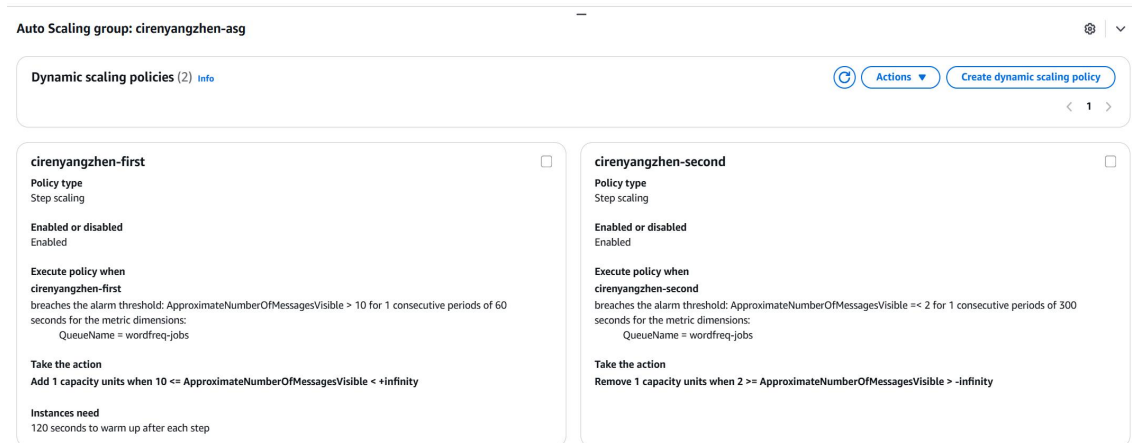


Fig.2. 7

The basic Auto-scaling has been implemented successfully.

3. Task C

After completing the deployment of the auto-scaling architecture, I conducted comprehensive load testing to verify system performance and the effectiveness of the scaling strategy. The tests used 120 text files provided by Blackboard as the standard load.

3.1 Test Steps

3.1.1 Environment Preparation: Clear all files in the processing bucket and upload 120 text files to the upload bucket. At the same time I stopped the created instance wordfreq-dev, ensuring that only instances created by Auto Scaling handle these tasks.

3.1.2 Test execution: Connect to the first instance created in the Auto Scaling group via SSH, and execute the command in PuTTY to start batch processing:

```
*** System restart required ***
Last login: Thu Dec  4 19:29:13 2025 from 137.222.229.4
ubuntu@ip-172-31-43-92:~$ aws s3 cp s3://ciren yangzhen-uploading s3://ciren yangzhen-processing --exclude "*" --include "*.txt" --recursive
copy: s3://ciren yangzhen-uploading/data/10.txt to s3://ciren yangzhen-processing/data/10.txt
copy: s3://ciren yangzhen-uploading/data/1.txt to s3://ciren yangzhen-processing/data/1.txt
copy: s3://ciren yangzhen-uploading/data/101.txt to s3://ciren yangzhen-processing/data/101.txt
copy: s3://ciren yangzhen-uploading/data/105.txt to s3://ciren yangzhen-processing/data/105.txt
copy: s3://ciren yangzhen-uploading/data/106.txt to s3://ciren yangzhen-processing/data/106.txt
copy: s3://ciren yangzhen-uploading/data/100.txt to s3://ciren yangzhen-processing/data/100.txt
copy: s3://ciren yangzhen-uploading/data/104.txt to s3://ciren yangzhen-processing/data/104.txt
copy: s3://ciren yangzhen-uploading/data/108.txt to s3://ciren yangzhen-processing/data/108.txt
```

Fig.2. 8

3.2 Initial Configuration Record Table

The initial configuration is shown in the Table 1 below.

Maximum Capacity	Warm-up Time	Cooldown Time	Scale-out Threshold	Scale-in Threshold	Total Processing Time(seconds)
5	120	300	10	2	500

Table 1

3.3 Extended Behavioral Observation and Recording

3.3.1 S3 Bucket: After the command was executed, all text files were quickly generated in the ciren yangzhen-processing bucket.

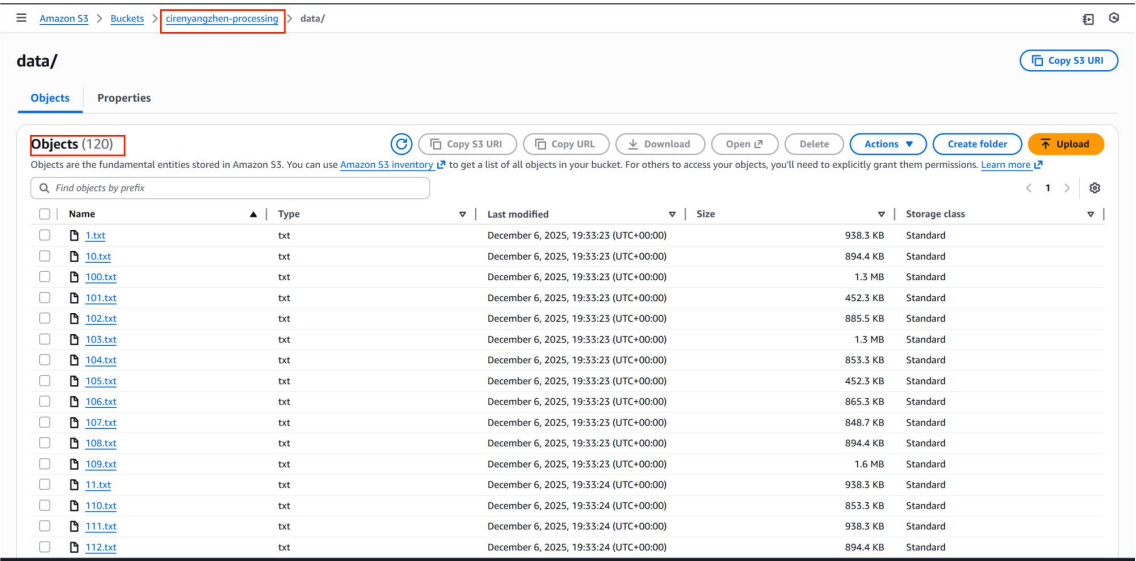


Fig.2. 9

3.3.2 Simple Queue Service: The dynamic process of the queue length decreasing from 120 to 0 during the test.

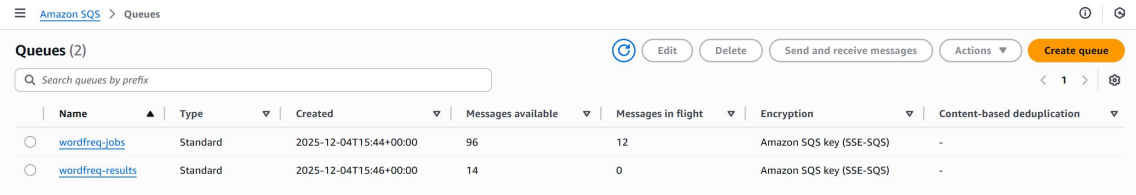


Fig.2. 10

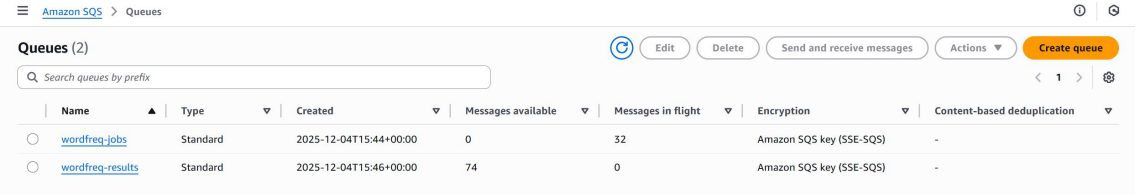


Fig.2. 11

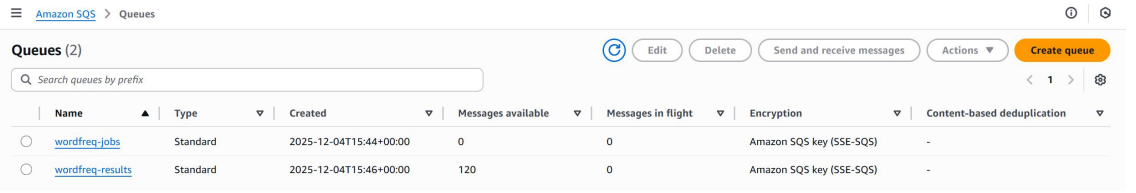


Fig.2. 12

3.3.3 Changes in the state of Auto Scaling group instances: It can be observed that the number of instances gradually increases from 1 to 5, and eventually returns to 1.

Auto Scaling group: cirenyangzhen-asg

Details | Integrations | Automatic scaling | **Instance management** | Instance refresh | Activity | Monitoring | Tags - moved

Instances (1)

Filter instances

Instance ID	Lifecycle	Instance type	Weighted c...	Launch tem...	Availability...	Health status	Protected fr...
i-09bf48703d83cfc0	InService	t2.micro	-	wordfreq-cirenyang	use1-az3 (us-e...	Healthy	

Fig.2. 13

Auto Scaling group: cirenyangzhen-asg

Details | Integrations | Automatic scaling | **Instance management** | Instance refresh | Activity | Monitoring | Tags - moved

Instances (2)

Filter instances

Instance ID	Lifecycle	Instance type	Weighted c...	Launch tem...	Availability...	Health status	Protected fr...
i-00eb7f811588f84a5	Pending	t2.micro	-	wordfreq-cirenyang	use1-az2 (us-e...	Healthy	
i-09bf48703d83cfc0	InService	t2.micro	-	wordfreq-cirenyang	use1-az3 (us-e...	Healthy	

Fig.2. 14

Auto Scaling group: cirenyangzhen-asg

Details | Integrations | Automatic scaling | **Instance management** | Instance refresh | Activity | Monitoring | Tags - moved

Instances (5)

Filter instances

Instance ID	Lifecycle	Instance type	Weighted c...	Launch tem...	Availability...	Health status	Protected fr...
i-00d38f8eef1b7d6b4	Pending	t2.micro	-	wordfreq-cirenyang	use1-az2 (us-e...	Healthy	
i-00eb7f811588f84a5	InService	t2.micro	-	wordfreq-cirenyang	use1-az2 (us-e...	Healthy	
i-044d5aa0e4ffbd75e	InService	t2.micro	-	wordfreq-cirenyang	use1-az3 (us-e...	Healthy	
i-09bf48703d83cfc0	InService	t2.micro	-	wordfreq-cirenyang	use1-az3 (us-e...	Healthy	
i-0e70877abcb4d136a	InService	t2.micro	-	wordfreq-cirenyang	use1-az6 (us-e...	Healthy	

Fig.2. 15

Auto Scaling group: cirenyangzhen-asg

Details | Integrations | Automatic scaling | **Instance management** | Instance refresh | Activity | Monitoring | Tags - moved

Instances (1)

Filter instances

Instance ID	Lifecycle	Instance type	Weighted c...	Launch tem...	Availability...	Health status	Protected fr...
i-044d5aa0e4ffbd75e	InService	t2.micro	-	wordfreq-cirenyang	use1-az3 (us-e...	Healthy	

Fig.2. 16

3.3.4 EC2: The figure shows the EC2 instance page, indicating that the wordfreq-dev instance was stopped before the test.

Instances (6) Info

Find Instance by attribute or tag (case-sensitive)

All states

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
wordfreq-dev	i-053289351df8f2826	Stopped	t2.micro	-	View alarms +	us-east-1a	-
	i-00eb7f811588f84a5	Terminated	t2.micro	-	View alarms +	us-east-1d	-
	i-00d38f8eef1b7d6b4	Terminated	t2.micro	-	View alarms +	us-east-1d	-
	i-09bf48703d83cfc0	Terminated	t2.micro	-	View alarms +	us-east-1e	-
	i-044d5aa0e4ffbd75e	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1e	ec2-100-26-209-6.com...
	i-0e70877abcb4d136a	Terminated	t2.micro	-	View alarms +	us-east-1b	-

Fig.2. 17

3.3.5 DynamoDB: After scanned the entire DynamoDB table wordfreq, I observed that there are 120 records, which corresponds to the 120 text files used in the test.

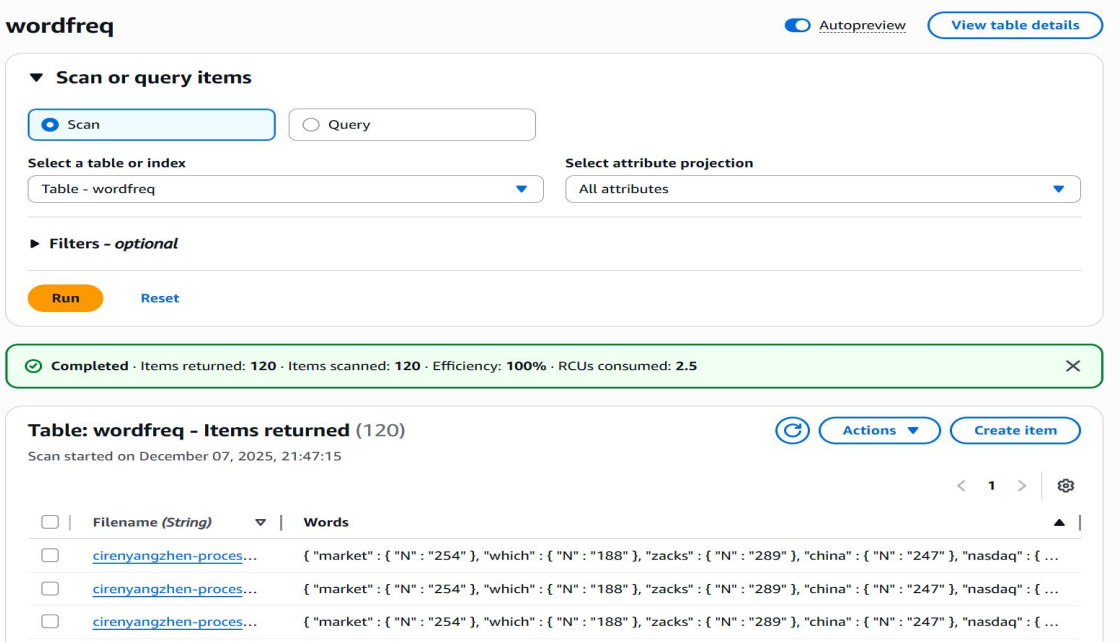


Fig.2. 18

3.3.6 Alarms: The figure displays the status changes of the two alarms, ciren yangzhen-first and ciren yangzhen-second during the text.

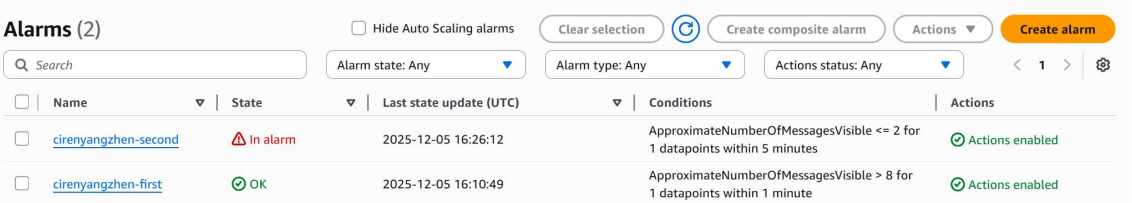


Fig.2. 19

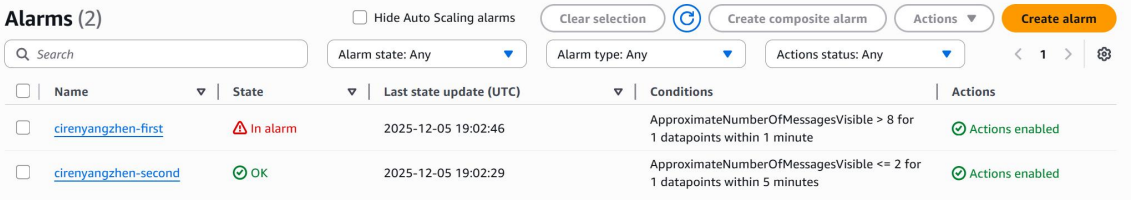


Fig.2. 20

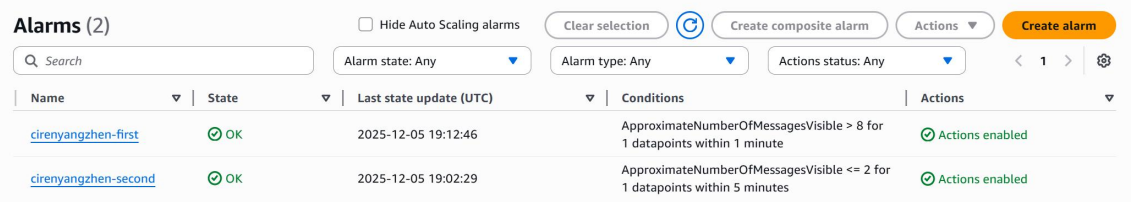


Fig.2. 21

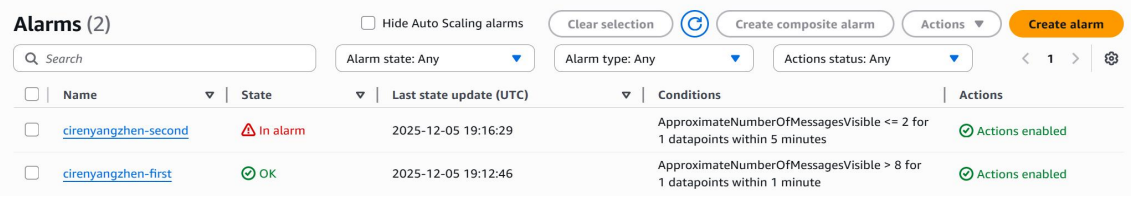


Fig.2. 22

3.4 Optimization of Scaling Operations

The following experiment employs a controlled variable method to test the total processing time for handling 120 files. Specific experimental variables and their effects are as follows:

3.4.1 Maximum Capacity: Increasing maximum capacity typically reduces total processing time. However, in this experiment, when Maximum Capacity exceeds 5, processing time remains largely unchanged due to instance warm-up delays, with only up to 5 instances being launched.

3.4.2 Warm-up Time: Shorter warm-up times allow new instances to begin processing tasks faster, reducing total processing time. Conversely, longer warm-up times increase total processing time.

3.4.3 Cooldown Time: Reducing cooldown from 300s to 100s increased processing time by approximately 5 minutes. This likely resulted from frequent scaling (expansion/contraction) due to overly short cooldown, causing "jitter." Conversely, excessively long cooldown may lead to idle instances and reduced resource utilization.

3.4.4 Scale-out Threshold: A threshold that is too low leads to over-allocation, increasing costs; a threshold that is too high delays scaling, increasing queue backlog time and consequently increasing processing time.

3.4.5 Scale-in Threshold: Increasing this threshold to 5 causes premature instance reduction, prolonging processing duration; an excessively high threshold may maintain too many instances, increasing costs.

Specific experimental data is shown in the Table 2 below:

Test Iteration	Maximum Capacity	Warm-up Time	Cooldown Time	Scale-out Threshold	Scale-in Threshold	Total Processing Time(seconds)
Baseline	5	120	300	10	2	500
2	5	120	300	10	5	771
3	5	120	300	10	1	507
4	4	120	300	10	2	756
5	6	120	300	10	2	528
6	5	120	100	10	2	840
7	5	120	300	12	2	752
8	5	120	300	8	2	861
9	5	110	300	10	2	488
10	5	130	300	10	2	825

Table 2

In conclusion, I find the configuration with the shortest processing time as Table 3.

Maximum Capacity	Warm-up Time	Cooldown Time	Scale-out Threshold	Scale-in Threshold	Total Processing Time(seconds)
5	110	300	10	2	488

Table 3

3.5 Evaluation of Different EC2 Instance Types

3.5.1 Experimental Results

Instance Type	Total Processing Time(seconds)
t2.micro	488
t2.small	460
t2.large	390
t3.micro	370
x2iedn.metal	520

Table 4

3.5.2 Instance Type Selection

(1) To analyze whether increasing CPU or memory within the same burstable family can linearly improve processing speed, t2.micro, t2.small, and t2.large were tested separately.

(2) To compare whether the new generation T3 is significantly faster than T2, t3.micro was tested.

(3) Finally, to verify whether stronger configurations always provide better performance, x2iedn.metal was selected for cross-architecture testing.

3.5.3 Result Analysis

(1) Memory capacity helps alleviate bottlenecks, but with diminishing returns:

The instance improvement from micro → small → large is slight, indicating that more memory can reduce I/O waiting or throttling, but it is not the main bottleneck.

(2) CPU performance has the greatest impact

Instances with more virtual CPUs, such as t3.micro and t2.large, consistently process files faster than the single-CPU instance t2.micro.

4.Task D

4.1 Improvement on Resilience and Availability

4.1.1 Multi-AZ Deployment: To enhance system availability in the event of a single point of failure, the ASG is extended to run in two Availability Zones. The ASG automatically replaces failed instances in the healthy Availability Zone, ensuring WordFreq remains available even if any single AZ fails, thus significantly improving the overall architecture's fault tolerance and high availability.

4.2 Long-term Backup and Data Protection

4.2.1 S3 Lifecycle Policies: S3 lifecycle rules are enabled for user-uploaded files and processing results files, automatically transferring older data to Glacier Instant Retrieval and ultimately to Glacier Deep Archive, achieving long-term and low-cost archival storage.

4.2.2 S3 Versioning: Versioning is enabled in buckets to prevent objects from being permanently lost due to accidental deletion or overwriting. Historical versions can be restored at any time, improving the security of critical data.

4.2.3 DynamoDB Continuous Backup with PITR: Store the metadata or application status information of word frequency results into DynamoDB and enable PITR. PITR effectively prevents the miswriting, deletion, or corruption of structured data.

4.3 Cost-effective and Efficient Processing

4.3.1 Change Instance Type to t4g.micro: Switch the instance from t2.micro to the more cost-effective t4g.micro. The Graviton2 architecture offers higher performance and lower cost, making it ideal for intermittent task processing scenarios like WordFreq.

4.3.2 Auto Scaling Minimum Capacity Modification: Set the minimum capacity of the Auto Scaling Group to 0, automatically shutting down all compute instances when there are no tasks. When file uploads cause the SQS queue depth to increase, ASG automatically scales up instances, achieving strict "on-demand use" and avoiding idle costs.

4.3.3 Use Spot Instances: Use EC2 Spot Instances to further reduce costs for non-real-time processing loads. Even if a Spot instance is reclaimed, SQS guarantees at least one message delivery, and newly launched instances can continue processing tasks, thus preventing data loss.

4.4 Prevent Unauthorized Access

4.4.1 Least Privilege IAM: Allow access only to specified S3 Buckets and SQS queues, and remove any irrelevant administrative permissions to reduce security risks and potential attack surfaces.

4.4.2 Private Subnet Placement: Deploy Worker EC2 in a private subnet, preventing instances from having public IPs and avoiding exposure to the internet at the physical network level.

Instances communicate with AWS services through a VPC Endpoint, without going through the public internet.

4.4.3 SG Default Deny: The security group uses a default denial policy, allowing only SQS, S3, and DynamoDB access outbound.

4.4.4 S3 Gateway Endpoint: To allow Worker EC2 instances in a private subnet to access S3 without going through the internet, an S3 Gateway Endpoint is added to the VPC. This endpoint provides a private network path to S3, improving security and access speed.

4.4.5 S3 Bucket Policies: Access is restricted through bucket policies, allowing only requests from specific IAM Roles and VPC Endpoints.

The whole structure is showed as figure .

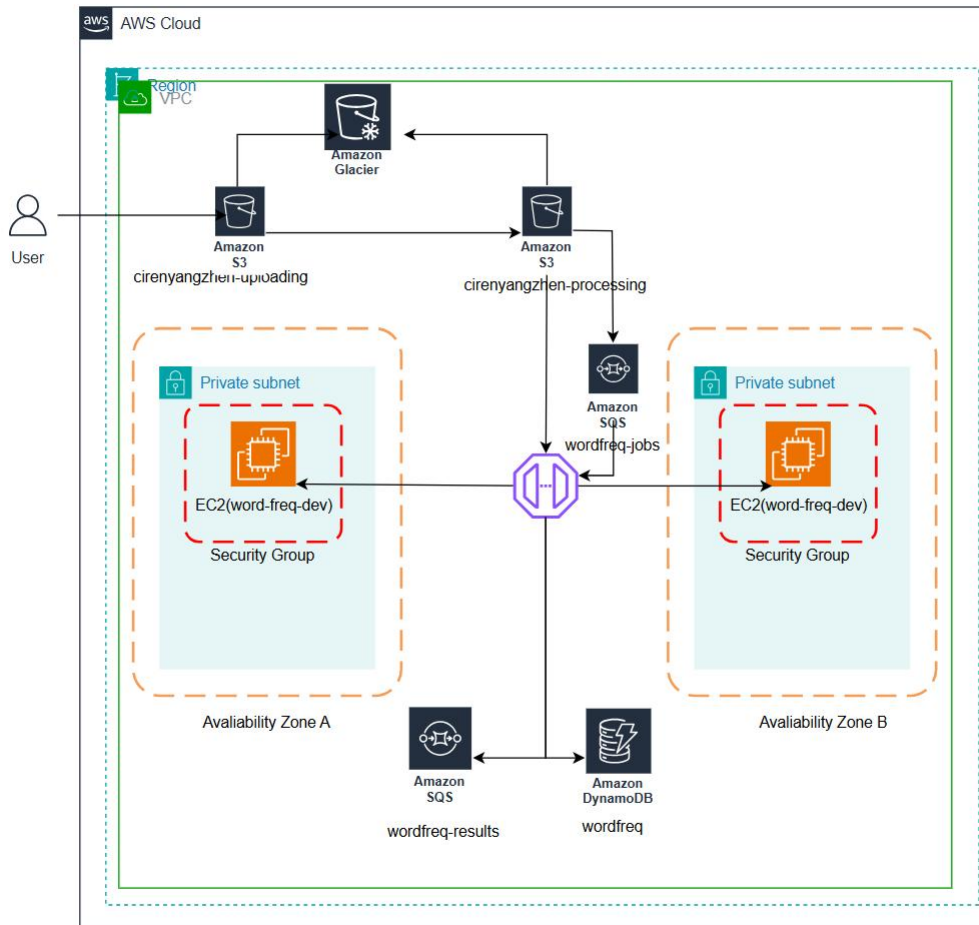


Fig.2. 23

5.Task E

5.1 Implementing Serverless Processing with AWS Lambda

The specific workflow is as follows: First, the user uploads a file to S3. Then, S3 triggers a Lambda function to perform word frequency statistics. Finally, the Lambda processing result is written back to DynamoDB.

5.1.1 Advantages:

- (1) Fully Serverless: No need to maintain any server components such as EC2, AMI, or Auto Scaling Group, reducing the operational burden.
- (2) Automatic Elasticity and Millisecond-Level Scaling: Lambda automatically scales horizontally based on the number of uploaded files, capable of processing hundreds or even thousands of files simultaneously without explicit configuration of scaling strategies.
- (3) Extremely Low Cost and Pay-Per-Call: Zero cost when there are no tasks; only the function execution time and request fees are charged, making it more cost-effective for WordFreq.
- (4) Built-in High Availability and Fault Tolerance: Lambda automatically redundant across multiple availability zones, eliminating the need to manage Multi-AZ.
- (5) Simplified security configuration: When used with VPC Endpoint, it can run entirely in a private

network without exposing EC2 and without requiring security group management.

5.2 Distributed Architecture Using Amazon EMR to Host Spark Cluster

The specific processing flow is as follows: Users upload files to S3, EMR starts Spark jobs through timed triggers, reads data from S3 and performs word frequency statistics in parallel within the cluster, and finally writes the results back to DynamoDB.

This architecture has the following advantages compared to the current version:

5.2.1 High-Performance Distributed Processing: Spark, through distributed in-memory computing, can split a single large file into multiple shards for parallel processing, resulting in significantly faster computation speeds than single-instance EC2.

5.2.2 Scalable Cluster: EMR supports automatic scaling, automatically increasing or decreasing worker nodes based on task size, achieving more efficient resource utilization.

5.2.3 Managed Cluster Operations: No need to manage the Hadoop or Spark environment yourself; EMR handles node deployment, health checks, and other maintenance tasks, providing better stability.

5.2.4 More suitable for complex data processing: Compared with the current single-machine version, Spark can easily support more complex text processing tasks such as data cleaning, sorting, and association analysis, providing more room for future expansion.

AWS account

email: re25226@bristol.ac.uk

Password: Cr@07002505

Reference:

1. Amazon Web Services, 2025.

<https://docs.aws.amazon.com/sagemaker/latest/dg/interface-vpc-endpoint.html>

3. Amazon Web Services, 2025. <https://aws.amazon.com/cn/ec2/instance-types/t2/>

4. Amazon Web Services, 2025. Instance Auto Scaling - Amazon EC2 Autoscaling - AWS

5. Amazon Web Services, 2025.

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Backup-and-Restore.html>

6. Amazon Web Services, 2025. <https://aws.amazon.com/cn/ec2/instance-types/t3/>